

# Foundations of Sparse attention transformer

[Sparse attention transformer]

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

## 1.0 Content Block

A decoder consists of an embedding layer, followed by multiple decoder layers, followed by an un-embedding layer. Each decoder consists of three major components: a causally masked self-attention mechanism, a cross-attention mechanism, and a feed-forward neural network. The decoder functions in a similar fashion to the encoder, but an additional attention mechanism is inserted which instead draws relevant information from the encodings generated by the encoders. This mechanism can also be called the encoder–decoder attention. Like the first encoder, the first decoder takes positional information and embeddings of the output sequence as its input, rather than encodings. The transformer must not use the current or future output to predict an output, so the output sequence must be partially masked to prevent this reverse information flow. This allows for autoregressive text generation. For decoding, all-to-all attention is inappropriate, because a token cannot attend to tokens not yet generated. Thus, the self-attention module in the decoder is causally masked. In contrast, the cross-attention mechanism attends to the output vectors of the encoder, which is computed before the decoder starts decoding. Consequently, there is no need for masking in the cross-attention mechanism. Schematically, we have:  $H' = \text{MaskedMultiheadAttention}(H, H, H)$   $\text{DecoderLayer}(H) = \text{FFN}(\text{MultiheadAttention}(H', H^E, H^E))$  where  $H^E$  is the matrix with rows being the output vectors from the encoder. The last decoder is followed by a final un-embedding layer to produce the output probabilities over the vocabulary. Then, one of the tokens is sampled according to the probability, and the decoder can be run again to produce the next token, etc., autoregressively generating output text.

## 2.0 Content Block

**KV caching** When an autoregressive transformer is used for inference, such as generating text, the query vector is different at each step, but the already-computed key and value vectors are always the same. The KV caching method saves the computed key and value vectors at each attention block, so that they are not recomputed at each new token. PagedAttention applies memory paging to KV caching. If a transformer is used with a baked-in prompt, such as ["You are a customer support agent..."], then the key and value vectors can be computed for the prompt, and saved on disk. The saving in compute is significant when the model is used for many short real-time interactions, such as in online chatbots. In general, when a user uses an autoregressive transformer to generate a continuation to a sequence of tokens, the model would first perform a forward-pass on this sequence, whereby the KV caches over this sequence are computed. This is called prefilling. Hyperscalers serving large Transformer models may use disaggregated inference, wherein prefilling and decoding are performed on separately specialized hardware.

## 3.0 Content Block

$\text{MultiheadAttention}(Q, K, V) = \text{Concat}_{i \in [n \text{ heads}]} (\text{Attention}(XW_i^Q, XW_i^K, XW_i^V)) W^O$  with Multi-Query Attention, there is just one  $W^K, W^V$ , thus:

#### 4.0 Content Block

Like earlier seq2seq models, the original transformer model used an encoder–decoder architecture. The encoder consists of encoding layers that process all the input tokens together one layer after another, while the decoder consists of decoding layers that iteratively process the encoder's output and the decoder's output tokens so far. The purpose of each encoder layer is to create contextualized representations of the tokens, where each representation corresponds to a token that "mixes" information from other input tokens via self-attention mechanism. Each decoder layer contains two attention sublayers: (1) cross-attention for incorporating the output of encoder (contextualized input token representations), and (2) self-attention for "mixing" information among the input tokens to the decoder (i.e. the tokens generated so far during inference time). Both the encoder and decoder layers have a feed-forward neural network for additional processing of their outputs and contain residual connections and layer normalization steps. These feed-forward layers contain most of the parameters in a transformer model.

#### 5.0 Content Block

The feedforward network (FFN) modules in a transformer are 2-layered multilayer perceptrons:  $\text{FFN}(x) = \phi(xW^{(1)} + b^{(1)})W^{(2)} + b^{(2)}$  where  $W^{(1)}$  and  $W^{(2)}$  are weight matrices and  $b^{(1)}$  and  $b^{(2)}$  are bias vectors, and  $\phi$  is its activation function. The original transformer used ReLU activation. The number of neurons in the middle layer is called intermediate size (GPT), filter size (BERT), or feedforward size (BERT). It is typically larger than the embedding size. For example, in both GPT-2 series and BERT series, the intermediate size of a model is 4 times its embedding size:  $d_{\text{ffn}} = 4d_{\text{emb}}$ .

#### 6.0 Content Block

**Un-embedding** An un-embedding layer is almost the reverse of an embedding layer. Whereas an embedding layer converts a token identifier into a vector, an un-embedding layer converts a vector into a probability distribution over tokens.

#### 7.0 Content Block

**Multimodality** Transformers can also be used/adapted for modalities (input or output) beyond just text, usually by finding a way to "tokenize" the modality. Multimodal models can either be trained from scratch, or by finetuning. A 2022 study found that transformers pretrained only on natural language can be finetuned on only 0.03% of parameters and become competitive with LSTMs on a variety of logical and visual tasks, demonstrating transfer learning. The LLaVA was a vision-language model composed of a language model (Vicuna-13B) and a vision model (ViT-L/14), connected by a linear layer. Only the linear layer is finetuned. Vision transformers adapt the transformer to computer vision by breaking down input images as a series of patches, turning them into vectors, and treating them like embedding vector of tokens in a standard transformer. Conformer and later Whisper follow the same pattern for speech recognition, first turning the speech signal into a spectrogram, which is then treated like an image, i.e. broken down into a series of patches, turned into vectors and treated like embedding vector of tokens in a standard transformer. Perceivers are a variant of transformers designed for multimodality. For image generation, notable architectures are DALL-E 1 (2021), Parti (2022), Phenaki (2023), and Muse (2023). Unlike later models, DALL-E is not a diffusion model. Instead, it uses a decoder-only transformer that autoregressively generates a text, followed by the token representation of an image, which is then converted by a variational autoencoder to an image. Parti is an encoder–decoder transformer, where the encoder processes a text prompt, and the decoder generates a token representation of an image. Muse is an encoder-only transformer that is trained to predict masked image tokens from unmasked image tokens. During generation, all input tokens are masked, and the highest-confidence predictions are included for the next iteration, until all tokens are predicted. Phenaki is a text-to-video model. It is a bidirectional

masked transformer conditioned on pre-computed text tokens. The generated tokens are then decoded to a video.